# MAD 1 WEEK 4

**Week 4: Models - Introduction to Databases**

---

### Persistent Storage

- **Definition:** Mechanism for storing data permanently on hard drives or storage devices.
- **In Web Applications:** Ensures data persists across user sessions and browser refreshes.
- **In MVC Architecture:** Pertains to data storage and retrieval within the Model component.

### Models

- Store and retrieve data within the model component.
- Represent data and associated business logic.

## RDBMS (Relational Database Management System)

- **Definition:** A database management system based on the relational model introduced by E.F. Codd.
- **Key Features:**
  - Data stored in **tables** (rows and columns).
  - **Primary Keys** uniquely identify records.
  - Use of **SQL (Structured Query Language)** for data operations.
  - Ensures **ACID properties**: Atomicity, Consistency, Isolation, Durability.
- **Advantages:**
  - Strong data integrity and consistency.
  - Supports complex queries with JOINs.
  - Standardized and widely adopted.
- **Disadvantages:**
  - Poor performance with large-scale, unstructured data.
  - Less flexible with schema changes.
- **Popular RDBMS:** MySQL, PostgreSQL, Oracle, Microsoft SQL Server.

### ACID Properties:

1. **Atomicity:** Transactions are all-or-nothing.

2. **Consistency:** Ensures data validity before and after transactions.
3. **Isolation:** Concurrent transactions don't affect each other.
4. **Durability:** Committed data remains even after system failures.

**Example:**

```
CREATE TABLE Students (

    ID INT PRIMARY KEY,

    Name VARCHAR(100),

    Age INT

);

INSERT INTO Students VALUES (1, 'John Doe', 22);

SELECT * FROM Students;
```

# NoSQL Databases

- **Definition:** Non-relational databases designed for large volumes of unstructured, semi-structured, or rapidly changing data.
- **Types of NoSQL Databases:**
  - **Document-Based:** Stores data in JSON/BSON format (e.g., MongoDB).
  - **Key-Value Stores:** Simple key-value pairs (e.g., Redis, DynamoDB).
  - **Column-Based:** Stores data in columns rather than rows (e.g., Cassandra, HBase).
  - **Graph-Based:** Focus on relationships between data (e.g., Neo4j).
- **Advantages:**
  - High scalability (horizontal scaling).
  - Flexible schema for dynamic data.
  - Optimized for large-scale, real-time applications.
- **Disadvantages:**
  - Lacks standardization (no universal query language).
  - Weaker data consistency compared to RDBMS.

Example:

```
use School;

db.Students.insertOne({ ID: 1, Name: 'Jane Doe', Age: 21 });
```

```
// Querying data

db.Students.find({ Name: 'Jane Doe' });
```

## Key Differences Between RDBMS and NoSQL

| Feature | RDBMS | NoSQL |
|---|---|---|
| Data Structure | Tables (rows & columns) | Documents, Key-Value, Graphs |
| Schema | Fixed schema | Dynamic schema |
| Scalability | Vertical | Horizontal |
| Query Language | SQL | Varies (MongoDB Query, etc.) |
| Transactions | Strong ACID compliance | BASE (Basically Available, Soft state, Eventually consistent) |
| Use Case | Structured data, transactions | Big data, real-time web apps |

### Need for Persistent Storage

- Retain data beyond program execution.
- Critical for applications like Gradebooks.

## Gradebook Example

- **Entities:**
  - Students: ID, Name, Address, etc.
  - Courses: ID, Name, Department, Year, etc.
  - Student-Course Relationship: Registration details.

## Spreadsheets

- Organize arbitrary data into Rows and Columns.
- Operations defined on Cells or Ranges.
- Support multiple interlinked sheets.
- Suitable for basic tabular data but limited in scalability and complex relationships.

LEC 4.3

## Relationships

- **Types:**

### 1. One-to-One (1:1)

- **Definition:** One record in a table is related to one record in another table.
- **Example:**
  - Each **person** has one **passport**.
  - Each student has a unique roll number.

### 2. One-to-Many (1:N)

- **Definition:** One record in a table is related to multiple records in another table.
- **Example:**
  - A **user** can have multiple **posts**.
  - A hostel has many students.

### 3. Many-to-One (N:1)

- **Definition:** Multiple records relate to one record. (This is just the inverse of One-to-Many.)
- **Example:**
  - Many **students** belong to one **class**.
  - Students register for multiple courses, and courses have many students.

## 4. Many-to-Many (M:N)

- **Definition:** Multiple records in one table relate to multiple records in another.
- **Example:**
    - Students can enroll in multiple **courses**, and each course can have multiple **students**.

- **Implementation:**
    - Using separate tables with unique IDs (Keys).
    - Reduces redundancy and maintains data integrity.

## In-Memory Data Structures

```
names = {0: 'Alice', 1: 'Bob', 2: 'Charlie'}
courses = {0: 'EE', 1: 'Mech', 2: 'Calculus'}
rels = [(0, 0), (1, 2), (2, 1)]
```

- **Issues:** Error-prone, not scalable, duplicate risks.

## Using Keys for Better Data Management

```
names = {0: 'Alice', 1: 'Bob', 2: 'Charlie'}

courses = {0: 'EE', 1: 'Mech', 2: 'Calculus'}

rels = [(0, 0), (1, 2), (2, 1)]
```

- **Benefits:** Reduces data entry errors, ensures uniqueness.

## Object-Oriented Approach

```
class Student:
    idnext = 0
    def __init__(self, name, hostel):
        self.name = name
        self.id = Student.idnext
        self.hostel = hostel
        Student.idnext += 1
```

- **Advantages:** Auto-initializes IDs, easy to add new fields.

## Data Persistence Techniques

- In-memory data is volatile; use:
  - **Pickle:** For serializing Python objects.
  - **CSV/TSV:** Simple structured data formats.
  - **Spreadsheets:** For basic tabular data.

LEC 4.4

## Relational Databases (SQL)

- Originated by IBM (~1970s).
- **Structure:**
  - Tables with Columns (fields) and Rows (records).
  - **Primary Key:** Uniquely identifies records.
  - **Foreign Key:** Establishes relationships between tables.

## SQL Queries

Example:

```
SELECT s.name
FROM Students s
JOIN StudentsCourses sc ON s.ID = sc.studentID
JOIN Courses c ON c.ID = sc.courseID
WHERE c.name = 'Calculus';
```

- WHERE c.name = 'Calculus';
- **Joins:**
  - **Inner Join:** Matches records from both tables.
  - **Cartesian Product:** Combines all possible record pairs.

## Unstructured Databases (NoSQL)

- Flexible with schema.
- Examples: MongoDB, CouchDB.
- Suitable for arbitrary data, but less validation.

## Entity-Relationship Diagrams (ERD)

- Visualize entities and their relationships.
- Tools: [Draw.io]